

Heaven's Light is Our Guide

Rajshahi University of Engineering and Technology



Department of Mechatronics Engineering

Course Title : Software Engineering
Course Code : CSE 2188
Experiment No : 01
Name of Experiment : Introduction to C++ Programming
Date of Experiment : 23 June 2026
Date of Submission : 7 July 2026

Submitted By	Submitted To
Name : TASNIMUL HASAN Roll : 2408020 Series : 24	Sumaya Ishrat Moyeen Assistant Professor Department of Mechatronics Engineering RUET Md. Hafiz Ahamed Assistant Professor Department of Mechatronics Engineering RUET

Experiment No: 01

Name of Experiment: Introduction to C++ Programming

1.1 Introduction

This experiment serves as a fundamental introduction to the C++ programming language. It covers the basic structure of a C++ program, how to display output to the console, and how to read input from a user. Two elementary problems are solved to demonstrate these concepts: a "Hello, World!" program and a program that interacts with the user to get their name, home, and hobby.

1.2 Objectives

1. To understand the basic syntax and structure of a C++ program.
2. To learn how to use the `cout` stream to print text to the console.
3. To learn how to use the `cin` stream to read user input.
4. To become familiar with compiling and running simple C++ programs.

1.3 Problems and Solutions

1.3.1 Problem 1: Hello World

This program demonstrates the minimal structure of a C++ application and how to print a line of text to the standard output.

Code

Listing 1.1: Hello World Program

```
1 #include <iostream>
2
3 int main() {
4     std::cout << "Hello World" << std::endl;
5     return 0;
6 }
```

Output

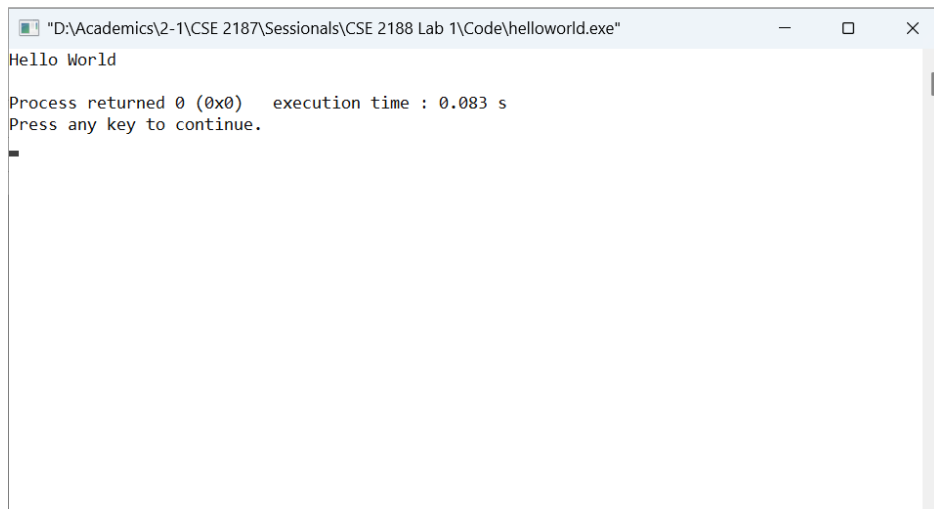


Figure 1.1: Output of the Hello World Program.

1.3.2 Problem 2: User Interaction

This program shows how to declare strings, prompt the user for input, read the input, and then display a formatted message using the information provided by the user.

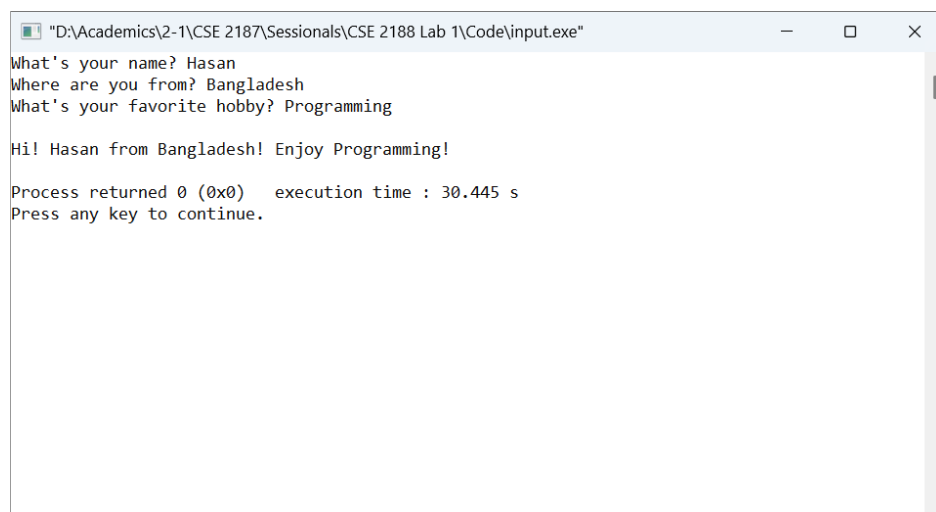
Code

Listing 1.2: User Interaction Program

```
1  #include <iostream>
2
3  int main() {
4      char name[100], hobby[100], home[100];
5
6      std::cout << "What's your name? ";
7      std::cin >> name;
8
9      std::cout << "Where are you from? ";
10     std::cin >> home;
11
12     std::cout << "What's your favorite hobby? ";
13     std::cin >> hobby;
```

```
14
15     std::cout << "\nHi! " << name << " from " <<
        home << "! Enjoy " << hobby << "!" << std::
        endl;
16     return 0;
17 }
```

Output



```
"D:\Academics\2-1\CSE 2187\Sessionals\CSE 2188 Lab 1\Code\input.exe"
What's your name? Hasan
Where are you from? Bangladesh
What's your favorite hobby? Programming

Hi! Hasan from Bangladesh! Enjoy Programming!

Process returned 0 (0x0)   execution time : 30.445 s
Press any key to continue.
```

Figure 1.2: Output of the User Interaction Program.

1.4 Conclusion

This experiment provided a solid foundation in C++ by demonstrating fundamental I/O operations. The first problem introduced basic console output with `std::cout`, while the second problem covered user input using `std::cin` with character arrays. A key observation from this approach is the limitation of `std::cin`, as it fails to capture any input after a whitespace character. This experience highlights the importance of selecting appropriate methods for handling different types of user input in C++ programming and reveals a potential pitfall of using `std::cin` for string-based data.

Heaven's Light is Our Guide

Rajshahi University of Engineering and Technology



Department of Mechatronics Engineering

Course Title : Software Engineering
Course Code : CSE 2188
Experiment No : 02
Name of Experiment : Study of Classes and Objects in C++
Date of Experiment : 7 July 2026
Date of Submission : 21 July 2026

Submitted By	Submitted To
Name : TASNIMUL HASAN Roll : 2408020 Series : 24	Sumaya Ishrat Moyeen Assistant Professor Department of Mechatronics Engineering RUET Md. Hafiz Ahamed Assistant Professor Department of Mechatronics Engineering RUET

Experiment No: 02

Name of Experiment: Study of Classes and Objects in C++

2.1 Introduction

This experiment transitions from procedural programming to the fundamentals of Object-Oriented Programming (OOP) in C++. It focuses on the core concepts of classes and objects. Through a series of problems, the experiment demonstrates how to define classes to model real-world entities, encapsulate data and behavior, and manage the lifecycle of objects using constructors and destructors. The problems involve creating a `Student` class to manage student records, a `Book` class to observe object creation and destruction, and a `Motor` class to perform encapsulated calculations.

2.2 Objectives

1. To understand the concept of classes and objects in C++.
2. To learn how to define a class with member variables and member functions (methods).
3. To implement constructors to initialize object state upon creation.
4. To understand the role of destructors in object lifecycle management.
5. To practice creating and manipulating multiple objects of a class.
6. To encapsulate data and functionality within a class to model real-world problems.

2.3 Problems and Solutions

2.3.1 Problem 1: Student Class

This program demonstrates the use of classes and objects in C++. It defines a `Student` class with a constructor to initialize student data and a method to display it. Multiple objects of the class are created to manage student records.

Code

```
1 #include <iostream>
2 #include <string>
3 using namespace std;
4
```

```
5 class Student {
6     string name;
7     int roll;
8     float cgpa;
9     string department;
10    int semester;
11
12 public:
13     // Constructor
14     Student(string n, int r, float c, string d, int
15             s)
16     {
17         name = n;
18         roll = r;
19         cgpa = c;
20         department = d;
21         semester = s;
22     }
23
24     // Display function
25     void display()
26     {
27         cout << "Name      : " << name << endl;
28         cout << "Roll      : " << roll << endl;
29         cout << "CGPA      : " << cgpa << endl;
30         cout << "Department : " << department <<
31             endl;
32         cout << "Semester   : " << semester << endl;
33         ;
34         cout << "-----" <<
35             endl;
```

```
32     }
33 };
34
35 int main()
36 {
37     // Multiple objects
38     Student s1("Hasan", 2408020, 3.16, "MTE", 3);
39     Student s2("Abdullah", 2403002, 3.65, "CSE", 3)
40         ;
41     Student s3("Umar", 2401018, 3.95, "EEE", 6);
42
43     // Print information
44     s1.display();
45     s2.display();
46     s3.display();
47
48     return 0;
49 }
```

Output

```
"D:\Academics\2-1\Lab Reports\CSE 2188\src\lab-02\code\lab-02.exe"
Name      : Hasan
Roll      : 2408020
CGPA      : 3.16
Department : MTE
Semester  : 3
-----
Name      : Abdullah
Roll      : 2403002
CGPA      : 3.65
Department : CSE
Semester  : 3
-----
Name      : Umar
Roll      : 2401018
CGPA      : 3.95
Department : EEE
Semester  : 6
-----
Process returned 0 (0x0)   execution time : 0.294 s
Press any key to continue.
```

Figure 2.1: Output of the Student Class Program.

2.3.2 Problem 2: Book Class with Constructor and Destructor

This program demonstrates the functionality of constructors and destructors in a C++ class. A Book class is created, where the constructor announces the creation of a book object and the destructor announces its removal.

Code

```
1 #include <iostream>
2 using namespace std;
3
4 class Book {
5     string title;
6     string author;
7     float price;
8
9 public:
10     Book(string t, string a, float p) {
11         title = t;
12         author = a;
13         price = p;
14         cout << title << " added to library." <<
15             endl;
16     }
17
18     void display() {
19         cout << "\nTitle   : " << title << endl;
20         cout << "Author   : " << author << endl;
21         cout << "Price    : " << price << endl;
22     }
23
24     ~Book() {
```

```
24         cout << title << " removed from library."
           << endl;
25     }
26 };
27
28 int main() {
29     Book b1("C++ Basics", "Bjarne", 450);
30     Book b2("Data Structures", "Mark", 550);
31     Book b3("Algorithms", "Thomas", 600);
32
33     b1.display();
34     b2.display();
35     b3.display();
36
37     return 0;
38 }
```

Output

```
"D:\Academics\2-1\Lab Reports\CSE 2188\src\lab-02\code\library.exe"
C++ Basics added to library.
Data Structures added to library.
Algorithms added to library.

Title : C++ Basics
Author : Bjarne
Price : 450

Title : Data Structures
Author : Mark
Price : 550

Title : Algorithms
Author : Thomas
Price : 600
Algorithms removed from library.
Data Structures removed from library.
C++ Basics removed from library.

Process returned 0 (0x0)   execution time : 0.099 s
Press any key to continue.
```

Figure 2.2: Output of the Book Class Program.

2.3.3 Problem 3: Motor Class for Electrical Calculations

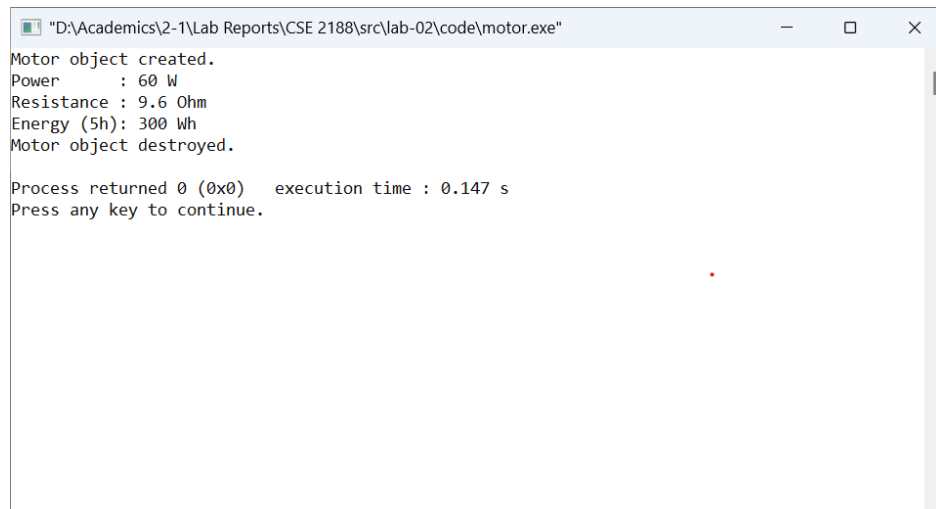
This program defines a Motor class to perform basic electrical calculations. It includes methods to calculate power, resistance, and energy, showcasing how classes can encapsulate data and related functionality.

Code

```
1 #include <iostream>
2 using namespace std;
3
4 class Motor {
5 private:
6     double voltage;
7     double current;
8
9 public:
10    // Constructor
11    Motor(double v, double c) {
12        voltage = v;
13        current = c;
14        cout << "Motor object created.\n";
15    }
16
17    double power() {
18        return voltage * current;
19    }
20
21    double resistance() {
22        if (current == 0) {
23            cout << "Resistance cannot be
24                calculated (Current = 0)." << endl;
25            return 0;
26        }
27    }
28 }
```

```
25         }
26         return voltage / current;
27     }
28
29     double energy(double hours) {
30         return power() * hours;
31     }
32
33     // Destructor
34     ~Motor() {
35         cout << "Motor object destroyed." << endl;
36     }
37 };
38
39 int main() {
40     Motor m1(24, 2.5);
41
42     cout << "Power      : " << m1.power() << " W"
43         << endl;
44     cout << "Resistance : " << m1.resistance() << "
45         Ohm" << endl;
46     cout << "Energy (5h): " << m1.energy(5) << " Wh
47         " << endl;
48
49     return 0;
50 }
```

Output



```
"D:\Academics\2-1\Lab Reports\CSE 2188\src\lab-02\code\motor.exe"
Motor object created.
Power : 60 W
Resistance : 9.6 Ohm
Energy (5h): 300 Wh
Motor object destroyed.

Process returned 0 (0x0) execution time : 0.147 s
Press any key to continue.
```

Figure 2.3: Output of the Motor Class Program.

2.4 Conclusion

This experiment provided practical experience with the fundamental principles of Object-Oriented Programming in C++. The implementation of the Student class demonstrated how to effectively group related data and functions to model a real-world entity. The Book class provided a clear illustration of an object's lifecycle, with constructors and destructors automatically managing initialization and cleanup. Finally, the Motor class showcased the power of encapsulation by bundling data (voltage, current) with methods that operate on that data (power, resistance calculations), creating a self-contained and reusable component. Collectively, these problems highlight the advantages of using classes for creating structured, modular, and maintainable code.

Heaven's Light is Our Guide

Rajshahi University of Engineering and Technology



Department of Mechatronics Engineering

Course Title : Software Engineering
Course Code : CSE 2188
Experiment No : 03
Name of Experiment : Study of Constructors and Destructors in C++
Date of Experiment : 21 July 2026
Date of Submission : 4 August 2026

Submitted By	Submitted To
Name : TASNIMUL HASAN Roll : 2408020 Series : 24	Sumaya Ishrat Moyeen Assistant Professor Department of Mechatronics Engineering RUET Md. Hafiz Ahamed Assistant Professor Department of Mechatronics Engineering RUET

Experiment No: 03

Name of Experiment: Study of Constructors and Destructors in C++

3.1 Introduction

Object-Oriented Programming (OOP) is a programming style that organizes data and functions into reusable units called classes. A class acts as a blueprint, while an object is an instance of that class. In this experiment, the main idea is to understand how classes help in modeling real-world entities such as students, books, or motors by combining data and behavior in one structure. This makes programs easier to understand, maintain, and extend compared with procedural programming.

A key feature of OOP is the use of constructors and destructors. Constructors are special member functions that initialize an object when it is created, while destructors are called automatically when an object is destroyed. Copy constructors are also important because they create a new object by copying the values of an existing object. These concepts are demonstrated in the experiment through simple class examples.

For example:

```
1  class Student {
2  public:
3      Student() { std::cout << "Constructor"; }
4      ~Student() { std::cout << "Destructor"; }
5  };
6
7  int main() {
8      Student s1;
9      return 0;
10 }
```

This experiment helps students observe how objects are created, copied, and removed automatically by the C++ runtime, which is an essential part of OOP.

3.2 Objectives

1. To implement default and parameterized constructors for object initialization.
2. To observe how copy constructors create duplicate objects from existing ones.

3. To understand the role of destructors in releasing object resources and ending object lifetime.

3.3 Problem

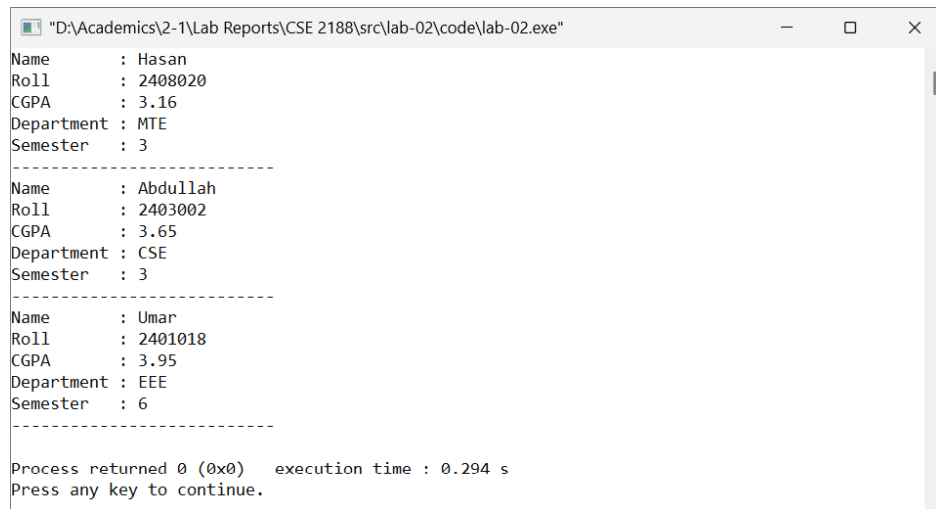
This program demonstrates the use of constructors, copy constructors, and destructors in a C++ class. It defines a Student class with default and parameterized constructors, then creates multiple objects to show object initialization and copy behavior.

Code

```
1  #include <iostream>
2
3  class Student {
4      int id;
5
6  public:
7      // Default Constructor
8      Student() {
9          id = 0;
10         std::cout << "Default Constructor Called"
11             << std::endl;
12     }
13
14     // Parameterized Constructor
15     Student(int x) {
16         id = x;
17         std::cout << "Parameterized Constructor
18             Called" << std::endl;
19     }
20
21     // Copy Constructor
22     Student(const Student &obj) {
```

```
21         id = obj.id + 1;
22         std::cout << "Copy Constructor Called" <<
           std::endl;
23     }
24
25     // Destructor
26     ~Student() {
27         std::cout << "Destructor Called for ID = "
           << id << std::endl;
28     }
29 };
30
31
32 int main() {
33     Student s1;           // Default Constructor
34
35     Student s2(2408020); // Parameterized
           Constructor
36
37     Student s3 = s2;      // Copy Constructor
38
39     Student s4(s3); // Copy Constructor
40
41     return 0;
42 }
```

Output



```
"D:\Academics\2-1\Lab Reports\CSE 2188\src\lab-02\code\lab-02.exe"
Name      : Hasan
Roll      : 2408020
CGPA      : 3.16
Department : MTE
Semester  : 3
-----
Name      : Abdullah
Roll      : 2403002
CGPA      : 3.65
Department : CSE
Semester  : 3
-----
Name      : Umar
Roll      : 2401018
CGPA      : 3.95
Department : EEE
Semester  : 6
-----
Process returned 0 (0x0)   execution time : 0.294 s
Press any key to continue.
```

Figure 3.1: Output showing default, parameterized, and copy constructor calls along with destructor execution.

3.4 Conclusion

This experiment helped in understanding the basic concepts of Object-Oriented Programming in C++. By working with classes and objects, it became clear how data and functions can be combined into a single unit to model real-world problems. The use of constructors, copy constructors, and destructors showed how objects are initialized, copied, and removed during program execution. Overall, the experiment demonstrated that OOP makes programs more organized, reusable, and easier to manage compared with simple procedural programming.

Heaven's Light is Our Guide

Rajshahi University of Engineering and Technology



Department of Mechatronics Engineering

Course Title : Software Engineering
Course Code : CSE 2188
Experiment No : 04
Name of Experiment : Study of Four Properties of Class in C++ Sensor Design
Date of Experiment : 4 August 2026
Date of Submission : 25 August 2026

Submitted By	Submitted To
Name : TASNIMUL HASAN Roll : 2408020 Series : 24	Sumaya Ishrat Moyeen Assistant Professor Department of Mechatronics Engineering RUET Md. Hafiz Ahamed Assistant Professor Department of Mechatronics Engineering RUET

Experiment No: 04

Name of Experiment: Study of Four Properties of Class in C++ Sensor Design

4.1 Introduction

This experiment presents a sensor system implemented in C++ that demonstrates the four main properties of a class: abstraction, encapsulation, inheritance, and polymorphism; an abstract base class named `Sensor` defines a common interface and hides implementation details while derived classes such as `TemperatureSensor` and `DistanceSensor` provide specific behavior, with abstraction shown by the base `Sensor` class and its pure virtual method, encapsulation enforced by keeping sensor values private and exposing controlled access methods, inheritance realized by deriving concrete sensor types from the abstract base class, and polymorphism exercised by using a base-class pointer array to call `display()` at runtime.

4.2 Objectives

1. To demonstrate the four properties of a class: abstraction, encapsulation, inheritance, and polymorphism.
2. To implement an abstract `Sensor` base class and derived concrete sensor classes.
3. To show encapsulation through private sensor values and public setter/getter methods.
4. To show runtime polymorphism using a base-class pointer array.
5. To explain how each part of the code reflects a class property.

4.3 Problem

This program demonstrates the four properties of a class through a sensor example.

Abstraction: The abstract base class `Sensor` defines a common interface for all sensors. It declares the pure virtual method `display()` and hides common details such as the sensor name. Concrete sensor classes must implement `display()`, which makes the design easier to extend and maintain.

Encapsulation: The derived classes `TemperatureSensor` and `DistanceSensor` store their values in private members (`temperature` and `distance`). These values cannot be changed directly from outside the class. Instead, public `setValue()` and `getValue()` methods provide controlled access.

Inheritance: TemperatureSensor and DistanceSensor inherit from Sensor. This relationship allows them to reuse the common sensor interface and share the name field from the base class.

Polymorphism: In main(), an array of Sensor* pointers stores addresses of the derived objects. The loop calls sensors[i]->display() on each pointer. Because display() is virtual, the correct derived implementation runs at runtime, showing polymorphic behavior.

Both derived classes also use overloaded setValue() methods to accept either integer or double input. While this is compile-time polymorphism, the main runtime polymorphism in the program comes from the virtual display() function.

Code

```
1  #include <iostream>
2  #include <string>
3
4  using namespace std;
5
6  // Abstract class (Abstraction)
7  class Sensor {
8  protected:
9      string name;
10
11 public:
12     Sensor(string n) {
13         name = n;
14     }
15
16     // Pure virtual function
17     virtual void display() = 0;
18
19     virtual ~Sensor() {}
20 };
21
```

```
22 // Inheritance + Encapsulation
23 class TemperatureSensor : public Sensor {
24 private:
25     double temperature;
26
27 public:
28     TemperatureSensor(string n) : Sensor(n) {
29         temperature = 0;
30     }
31
32     // Function overloading
33     void setValue(int t) {
34         temperature = t;
35     }
36
37     void setValue(double t) {
38         temperature = t;
39     }
40
41     double getValue() {
42         return temperature;
43     }
44
45     // Polymorphism
46     void display() override {
47         cout << name << " Temperature: "
48             << temperature << " degree C" << endl;
49     }
50 };
51
52 class DistanceSensor : public Sensor {
```

```
53 private:
54     double distance;
55
56 public:
57     DistanceSensor(string n) : Sensor(n) {
58         distance = 0;
59     }
60
61     // Function overloading
62     void setValue(int d) {
63         distance = d;
64     }
65
66     void setValue(double d) {
67         distance = d;
68     }
69
70     double getValue() {
71         return distance;
72     }
73
74     void display() override {
75         cout << name << " Distance: "
76             << distance << " cm" << endl;
77     }
78 };
79
80 int main() {
81
82     TemperatureSensor temp("LM35");
83     DistanceSensor dist("Ultrasonic");
```

```
84
85     temp.setValue(36.5);    // double version
86     dist.setValue(125);    // int version
87
88     // Array of pointers (Pointer + Array)
89     Sensor* sensors[2];
90
91     sensors[0] = &temp;
92     sensors[1] = &dist;
93
94     cout << "=== Sensor Readings ===\n\n";
95
96     // Polymorphism in action
97     for (int i = 0; i < 2; i++) {
98         sensors[i]->display();
99     }
100
101     return 0;
102 }
```

Output

```
=== Sensor Readings ===
LM35 Temperature: 36.5 degree C
Ultrasonic Distance: 125 cm
```

Figure 4.1: Program output showing temperature and distance sensor readings via polymorphic base-class pointers.

4.4 Conclusion

The lab demonstrated the four core properties of a class in C++: abstraction, encapsulation, inheritance, and polymorphism. The abstract Sensor class provided a reusable interface, while derived classes for temperature and distance encapsulated their own data and behavior. Using a Sensor* array to call display() showed runtime polymorphism and made the code

flexible for future sensor types. Overall, the experiment reinforced how class properties help structure code, reduce duplication, and support maintainable designs.